

Template MPC for Switched LPV Robotic Systems: Learned Contact Scheduling with Structured Dynamics

Jiangnan Zhang
Dartmouth College

Abstract—We present a four-layer control architecture for legged robots in which the *only* external specification is a six-dimensional body-pose command (position and orientation of the center of mass relative to the support polygon). From this command, the system autonomously determines where to place its feet, what dynamics govern each contact phase, and what joint torques to apply—with no pre-defined gait table and no analytical dynamics model.

The four layers are: (i) a *user command* $s \in \mathbb{R}^6$ specifying the desired body pose; (ii) a *Footstep Planner Network* that maps the command and current state to a time-stepped sequence of desired end-effector positions over the MPC horizon, from which the contact mode at each timestep is derived via the signed distance to the contact surface; (iii) per-mode *Euler-Lagrangian Networks* (EL-Nets) that provide smooth linearized dynamics while preserving symmetric positive-definite inertia, energy conservation, and passivity; and (iv) a *DARE-cached MPC solver* that assembles a complete time-varying quadratic program from these pieces and solves it via precomputed Riccati gains.

The MPC problem is not passed through a monolithic solver; it is *assembled* at each timestep from four independently sourced components: the reference (from the user command), the contact constraints and scheduling variable (from the Planner), the dynamics matrices (from the EL-Nets), and the feedback gains (from the DARE cache). This generalizes the flat-parameterized TinyMPC framework [1] from quadrotors (single mode, analytic dynamics) to switched linear parameter-varying (LPV) systems with learned dynamics and learned contact scheduling.

I. INTRODUCTION

Model-predictive control (MPC) is a central tool for controlling robots subject to state and input constraints [2]. For systems with persistent contacts—such as quadrupedal locomotion—the dominant paradigm is *whole-body control* (WBC) [3]: a pre-defined gait scheduler selects the contact sequence, and a quadratic program computes joint torques that respect contact constraints, friction cones, and dynamic feasibility. The MIT Cheetah [3] and similar platforms demonstrate the effectiveness of this approach at moderate speeds with analytical rigid-body dynamics.

Two limitations of the standard WBC pipeline motivate our work. First, the contact sequence is *pre-defined*—typically a periodic gait table (trot, bound, gallop)—which prevents reactive adaptation to unstructured terrain. Second, the dynamics model is *analytical*, requiring accurate knowledge of inertial parameters, joint friction, and contact geometry. For complex morphologies or soft contacts, such models are difficult to obtain.

In prior work [1], we showed that for quadrotors—a contact-free, differentially flat system—the DARE solution can be cached over a four-dimensional flat parameter $\alpha = (a_x, a_y, a_z, \psi)$, reducing online MPC to ~ 260 arithmetic operations. The key enabler was that differential flatness compresses the scheduling variable from $\mathbb{R}^n$ to $\mathbb{R}^4$.

This paper generalizes the construction to *switched linear parameter-varying* (LPV) systems—the natural abstraction for robots that make and break contacts. We propose a four-layer architecture in which each layer has exactly one responsibility and one mathematical type:

Layer	Role	Input	Output
1. Command	Intention	—	$s \in \mathbb{R}^6$
2. Planner	Foot placement	$x, s, \mathcal{S}$	$p_i^{\text{des}}(t)$
3. MPC	Torques	σ, α, x	$\tau \in \mathbb{R}^m$
4. Low-level	Execution	τ	motor cmds

Each layer *reduces* the problem: 6 numbers of intention $\rightarrow C \times 3 \times N$ numbers of contact geometry $\rightarrow m$ numbers of torques $\rightarrow$ hardware. No layer crosses the smooth/nonsmooth boundary: the Footstep Planner (Layer 2) handles all discrete contact decisions; the MPC solver (Layer 3) sees only smooth dynamics.

The architecture mirrors the whole-body control (WBC) pipeline [3] but replaces its two pre-specified components:

Component	WBC [3]	Ours
User input	Body-pose cmd	Same
Contact sched.	Pre-defined gait	Learned
Dynamics	Analytical RBD	EL-Net
Feedback	QP (online)	DARE (offline)
Pre-spezif.	Gait + dynamics	Nothing

The only external input is the user’s body-pose command $s \in \mathbb{R}^6$ —six numbers. Everything else is either learned (Footstep Planner, EL-Net) or derived from classical control theory (DARE cache). Pre-defined gaits (trot, walk, gallop) can serve as a bootstrapping curriculum for training the Footstep Planner, enabling a smooth transition from classical to learned scheduling.

Contributions

- 1) A *four-layer architecture* (user command $\rightarrow$ footstep planner $\rightarrow$ smooth MPC $\rightarrow$ low-level control) in which

the only external specification is a six-dimensional body-pose command. The MPC problem at each timestep is *assembled*—not passed through a monolithic solver—from four independently sourced components: reference, contact schedule, dynamics, and feedback gains (Section III-A).

- 2) A *switched LPV* formulation with precise geometric primitives (world frame, contact surface, signed distance), contact modes as derived quantities, and a scheduling variable that is exactly determined by the relative contact geometry (Section II).
- 3) A *Footstep Planner Network* that outputs a time-stepped sequence of desired end-effector positions over the MPC horizon, from which per-timestep contact modes are derived—enabling time-varying mode sequences within a single predictive horizon (Section III-B).
- 4) *EL-Nets*: Euler–Lagrangian structured neural networks for per-mode dynamics with guaranteed SPD inertia, energy conservation, and passivity. Gravitational potential is analytically known; only internal potentials are learned (Section III-C).
- 5) A generalized equivalence chain (Theorem 1) and sufficient conditions (Proposition 3), with the quadrotor result of [1] recovered as a single-mode special case (Section IV).

II. PROBLEM FORMULATION

A. Geometric Primitives

We begin by establishing the geometric objects needed to define contacts.

Definition 1 (World Frame and Contact Surface). A world frame $\mathcal{W}$ is an inertial reference frame with a distinguished “up” direction e_z . The gravity vector is $\mathbf{g} = -g\mathbf{e}_z \in \mathbb{R}^3$ with $g > 0$. A contact surface $\mathcal{S}$ is a subset of $\mathcal{W}$ represented by a signed distance function $\phi : \mathbb{R}^3 \rightarrow \mathbb{R}$, with $\mathcal{S} = \{\mathbf{p} \in \mathbb{R}^3 : \phi(\mathbf{p}) = 0\}$ and $\phi(\mathbf{p}) > 0$ for points above $\mathcal{S}$.

For flat ground, $\phi(\mathbf{p}) = p_z$ and $\mathcal{S} = \{p_z = 0\}$. For general terrain, ϕ is a signed distance field (SDF) that may be provided by a depth sensor or terrain map.

Definition 2 (Contact Points and Contact Mode). A robot has C potential contact points (e.g., feet, fingertips) whose positions in $\mathcal{W}$ are functions of the generalized coordinates: $\mathbf{p}_i(\mathbf{q}) \in \mathbb{R}^3$, $i = 1, \dots, C$. The contact mode is the binary vector

$$\sigma \in \{0, 1\}^C, \quad \sigma_i = \begin{cases} 1 & \text{if } \phi(\mathbf{p}_i(\mathbf{q})) = 0, \\ 0 & \text{if } \phi(\mathbf{p}_i(\mathbf{q})) > 0. \end{cases} \quad (1)$$

The set of reachable modes is $\mathcal{I} \subseteq \{0, 1\}^C$ with $|\mathcal{I}| \leq 2^C$.

Remark 1 (Contact Mode Is Derived, Not Prescribed). The contact mode σ is a consequence of the end-effector positions relative to $\mathcal{S}$, not an independent control input. To break contact i , one moves $\mathbf{p}_i$ so that $\phi(\mathbf{p}_i) > 0$; to make contact, one moves $\mathbf{p}_i$ onto $\mathcal{S}$. This observation motivates the Footstep Planner architecture in Section III-B.

B. Robot State Decomposition

We decompose the robot state into three components, each with a distinct role in the scheduling variable.

Definition 3 (State Decomposition). The robot state is partitioned as

$$\mathbf{x} = (\mathbf{x}_{\text{com}}, \mathbf{x}_{\text{ee}}, \mathcal{W}), \quad (2)$$

where:

- 1) $\mathbf{x}_{\text{com}}$ is the center-of-mass (COM) state: position, velocity, orientation, and angular velocity of the floating base, expressed in $\mathcal{W}$.
- 2) $\mathbf{x}_{\text{ee}} = (\mathbf{p}_1, \dots, \mathbf{p}_C)$ is the end-effector configuration: positions of all C potential contact points in $\mathcal{W}$.
- 3) $\mathcal{W}$ provides the gravity vector $\mathbf{g}$ and the contact surface $\mathcal{S}$.

The absolute COM position in $\mathcal{W}$ does not affect the linearized dynamics (by translational invariance of the equations of motion). What matters is the *relative geometry*:

Definition 4 (Scheduling Variable). For a given contact mode σ , the scheduling variable $\alpha_\sigma \in \mathbb{R}^{d_\sigma}$ consists of:

- 1) **Active foot positions relative to COM**, projected onto $\mathcal{S}$: for each i with $\sigma_i = 1$, the 2D position $(\mathbf{p}_i - \mathbf{p}_{\text{com}})_{xy}$ on the contact surface. Contributes $2|\sigma|$ parameters, where $|\sigma| = \sum_i \sigma_i$ is the number of active contacts.
- 2) **COM skew**: the 2D displacement of $\mathbf{p}_{\text{com}}$ relative to the centroid of the support polygon formed by active contact points, projected onto $\mathcal{S}$. Contributes 2 parameters.

The total dimension is $d_\sigma = 2|\sigma| + 2$.

Proposition 1 (Exact Parameterization). The scheduling variable α_σ is exactly determined: given σ and α_σ , the linearized dynamics $A_\sigma(\alpha_\sigma)$, $B_\sigma(\alpha_\sigma)$ are uniquely specified (up to the learned dynamics model). Conversely, every component of α_σ affects the linearization—no component is redundant.

Proof sketch. The relative foot-COM positions determine the contact Jacobian J_σ and the holonomic constraint structure, which enter the linearization through the constrained equations of motion. The COM skew determines the equilibrium contact forces (via static balance), which affect the linearization of the friction cone and gravity torques. Removing any component leaves the linearization underdetermined at non-degenerate configurations. $\square$

Table I lists the scheduling variable dimension for representative systems.

C. Switched LPV System

Definition 5 (Switched LPV System). A switched linear parameter-varying system is the tuple $(\mathcal{I}, \{A_\sigma, B_\sigma\}_{\sigma \in \mathcal{I}}, \alpha, \mathcal{W}, \mathcal{S})$ where:

- $\mathcal{I} \subseteq \{0, 1\}^C$ is the set of reachable contact modes.
- For each $\sigma \in \mathcal{I}$, the per-mode dynamics

$$\dot{\mathbf{x}} = A_\sigma(\alpha_\sigma) \mathbf{x} + B_\sigma(\alpha_\sigma) \mathbf{u} \quad (3)$$

TABLE I
SCHEDULING VARIABLE DIMENSION $d_\sigma = 2|\sigma| + 2$ FOR REPRESENTATIVE
ROBOTIC SYSTEMS IN A TYPICAL CONTACT MODE.

System	C	Mode σ	$ \sigma $	d_σ
Quadrotor	0	(free flight)	0	4*
Biped (single)	2	(1, 0)	1	4
Biped (double)	2	(1, 1)	2	6
Quadruped (trot)	4	(1, 0, 1, 0)	2	6
Quadruped (stand)	4	(1, 1, 1, 1)	4	10

*Quadrotor uses differential flatness: $\alpha = (a_x, a_y, a_z, \psi)$ [1].

are smooth functions of the scheduling variable $\alpha_\sigma \in \mathbb{R}^{d_\sigma}$.

- The contact mode $\sigma(t)$ switches according to (1), driven by end-effector motion relative to $\mathcal{S}$.

Proposition 2 (Quadrotor as Single-Mode Special Case). *The quadrotor system of [1] is a switched LPV system with $\mathcal{I} = \{0\}$ (single mode, no contacts), $\mathcal{S} = \emptyset$, and analytically known dynamics $A(\alpha)$, $B(\alpha)$ from differential flatness with $\alpha = (a_x, a_y, a_z, \psi) \in \mathbb{R}^4$.*

D. Problem Statement

Given a switched LPV robotic system $(\mathcal{I}, \{A_\sigma, B_\sigma\}, \alpha, \mathcal{W}, \mathcal{S})$ and a user command $s \in \mathbb{R}^6$ (desired body pose relative to the support polygon):

- 1) **Contact planning**: Determine a time-stepped sequence of desired end-effector positions $\{p_i^{\text{des}}(t)\}_{t=0}^{N-1}$, from which the per-timestep contact mode $\sigma(t)$ and scheduling variable $\alpha(t)$ are derived.
- 2) **Dynamics**: For each timestep t and mode $\sigma(t)$, obtain the smooth linearization $A_{\sigma(t)}(\alpha(t))$, $B_{\sigma(t)}(\alpha(t))$.
- 3) **MPC assembly**: Assemble the time-varying quadratic program from the reference (derived from s), the contact constraints and scheduling (from step 1), and the dynamics (from step 2).
- 4) **Solve and execute**: Solve the assembled QP using DARE-cached gains and apply the resulting torques to the robot.
- 5) **Efficiency**: Total online computation must be suitable for real-time embedded deployment. The only external input is s —six numbers.

III. METHOD

A. Architecture Overview

The four-layer pipeline is illustrated in Fig. 1. Each layer has one responsibility, one input type, and one output type. Data flows strictly downward; no layer communicates with a non-adjacent layer.

- 1) **User command** $s \in \mathbb{R}^6$: desired body pose relative to the support polygon. This is the only external input. Within the MPC horizon, the controller tracks s as a constant reference until the next command arrives.
- 2) **Footstep Planner** (learned, Section III-B): maps $(x_t, s, \mathcal{S})$ to a time-stepped sequence of desired end-effector positions $p_i^{\text{des}}(t) \in \mathbb{R}^3$ for $t = 0, \dots, N-1$.

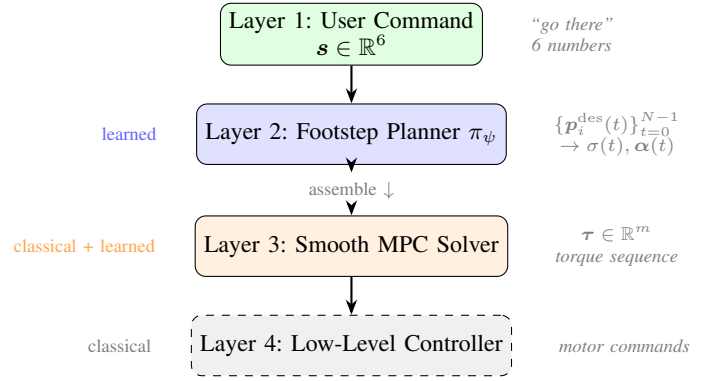


Fig. 1. Four-layer architecture. Data flows strictly downward: 6 numbers of intention $\rightarrow C \times 3 \times N$ numbers of contact geometry $\rightarrow m$ torques $\rightarrow$ hardware. The MPC problem at Layer 3 is assembled from the reference (Layer 1), contact schedule (Layer 2), dynamics (EL-Net), and gains (DARE cache).

The contact mode $\sigma(t)$ and scheduling variable $\alpha(t)$ are derived at each timestep via (1).

- 3) **Smooth MPC solver** (Section III-D): assembles a time-varying QP from the reference, the per-timestep $(\sigma(t), \alpha(t))$, the EL-Net dynamics (Section III-C), and the DARE-cached gains; then solves it via ADMM.
- 4) **Low-level controller** (classical): executes the resulting joint torques on hardware via PD tracking or inverse dynamics.

B. Footstep Planner Network

User command. The operator provides a body-pose command $s \in \mathbb{R}^6$, specifying the desired position and orientation of the center of mass relative to the support polygon (three translational, three rotational degrees of freedom). This is the *only* external specification. Within the MPC predictive horizon, the controller tracks s as a constant reference until the next command arrives.

Time-stepped foot placement sequence. The Footstep Planner is a neural network that outputs a foot-placement sequence over the full MPC horizon:

$$\pi_\psi : \mathbb{R}^n \times \mathbb{R}^6 \times \mathcal{S} \rightarrow \mathbb{R}^{N \times C \times 3}, \quad (4)$$

mapping the current state x , user command s , and contact surface $\mathcal{S}$ to desired end-effector positions $p_i^{\text{des}}(t) \in \mathbb{R}^3$ for each contact point $i = 1, \dots, C$ and each timestep $t = 0, \dots, N-1$ of the predictive horizon.

The contact mode at each timestep is derived:

$$\sigma_i(t) = \begin{cases} 1 & \text{if } \phi(p_i^{\text{des}}(t)) = 0, \\ 0 & \text{if } \phi(p_i^{\text{des}}(t)) > 0. \end{cases} \quad (5)$$

This means the Planner outputs a complete *contact schedule with timing*: it specifies not only which feet are in contact, but when each contact is made or broken, and where each foot lands. The mode sequence $\{\sigma(t)\}_{t=0}^{N-1}$ can vary within a single MPC horizon—for example, a trot step may begin with $\sigma = (1, 0, 1, 0)$ and transition to $(0, 1, 0, 1)$ midway through the horizon.

Why positions, not modes. This formulation resolves a fundamental issue with directly predicting σ : a bare mode classifier cannot express *where* a foot should land (making contact) or *how far* it should lift (breaking contact). By outputting continuous positions, the Planner simultaneously specifies the contact mode, the contact geometry, and the swing-leg trajectory.

Relationship to WBC gait scheduling. In the MIT Cheetah pipeline [3], the gait scheduler is a finite-state machine that cycles through pre-defined contact modes at fixed timing (e.g., trot: alternating diagonal pairs at a fixed frequency). Our Footstep Planner replaces this with a learned policy that adapts both *timing* and *placement* to terrain and velocity. Pre-defined gaits can serve as a *bootstrapping curriculum*: initialize the Planner to reproduce a trot gait, then fine-tune via reinforcement learning on varied terrain.

Training. The Planner is trained via policy gradient methods (REINFORCE [12] or PPO [13]), with the tracking cost of the downstream DARE-cached MPC as the reward signal. The EL-Net and DARE cache are frozen during Planner training.

Remark 2 (No Soft Mode Mixing). *One might consider a differentiable relaxation: replace the hard mode selection (5) with soft weights w_σ and blend dynamics as $A_{\text{eff}} = \sum_\sigma w_\sigma A_\sigma$. This is physically meaningless: a convex combination of “foot-on-ground” and “foot-in-air” dynamics does not correspond to any realizable physical system. The DARE solution of the blended system does not produce a useful controller for either mode. This is fundamentally different from soft attention in sequence models, where convex combinations of representations are semantically meaningful. The discrete mode boundary is precisely where end-to-end differentiability breaks—and the reason for the two-stage training architecture.*

C. EL-Net: Euler–Lagrangian Structured Dynamics

Within each contact mode σ , the dynamics are smooth. We learn them using a neural network that preserves the Euler–Lagrange structure of mechanical systems.

Definition 6 (EL-Net). *For a mechanical system with generalized coordinates $\mathbf{q} \in \mathbb{R}^{n_q}$, an Euler–Lagrangian Network (EL-Net) parameterizes:*

- 1) **Inertia matrix** $M_\theta(\mathbf{q}) \in \mathbb{S}_{++}^{n_q}$ via Cholesky factorization: $M_\theta = L_\theta(\mathbf{q}) L_\theta(\mathbf{q})^\top$, where L_θ is a lower-triangular matrix-valued neural network with positive diagonal entries. This guarantees $M_\theta \succ 0$ by construction.
- 2) **Potential energy** $V(\mathbf{q}) = V_{\text{grav}}(\mathbf{q}; \mathbf{g}, \mathcal{W}) + V_\theta(\mathbf{q})$, where $V_{\text{grav}}(\mathbf{q}) = -m \mathbf{g}^\top \mathbf{p}_{\text{com}}(\mathbf{q})$ is the known gravitational potential (from $\mathcal{W}$) and $V_\theta(\mathbf{q})$ is a scalar-valued neural network capturing internal elastic and dissipative potentials.
- 3) **Coriolis matrix** $C_\theta(\mathbf{q}, \dot{\mathbf{q}})$ derived from M_θ via the Christoffel symbols of the first kind:

$$[C_\theta]_{ij} = \sum_{k=1}^{n_q} \frac{1}{2} \left(\frac{\partial [M_\theta]_{ij}}{\partial q_k} + \frac{\partial [M_\theta]_{ik}}{\partial q_j} - \frac{\partial [M_\theta]_{jk}}{\partial q_i} \right) \dot{q}_k. \quad (6)$$

This construction guarantees that $\dot{M}_\theta - 2C_\theta$ is skew-symmetric, which is the passivity condition.

The equations of motion within mode σ are:

$$M_\theta(\mathbf{q}) \ddot{\mathbf{q}} + C_\theta(\mathbf{q}, \dot{\mathbf{q}}) \dot{\mathbf{q}} + \nabla_{\mathbf{q}} V(\mathbf{q}) = B_\sigma \mathbf{u} + J_\sigma(\mathbf{q})^\top \boldsymbol{\lambda}, \quad (7)$$

where B_σ is the actuator input matrix (known from the kinematic chain and mode σ), $J_\sigma(\mathbf{q})$ is the contact Jacobian for the active contact points in mode σ , and $\boldsymbol{\lambda}$ are the contact forces (eliminated by the holonomic constraints).

Guaranteed properties.

- 1) *SPD inertia*: $M_\theta \succ 0$ by the Cholesky parameterization.
- 2) *Energy conservation*: the skew-symmetry of $\dot{M}_\theta - 2C_\theta$ is automatic from the Christoffel construction (6).
- 3) *Passivity*: follows from energy conservation and positive-definite dissipation [5].

Linearization. At an equilibrium $(\mathbf{q}^*, \dot{\mathbf{q}}^* = 0)$ within mode σ with scheduling variable $\boldsymbol{\alpha}$, the linearized dynamics are:

$$\begin{aligned} A_\sigma(\boldsymbol{\alpha}) &= \begin{pmatrix} 0 & I \\ -M_\theta^{-1} H_V & -M_\theta^{-1} C_\theta \end{pmatrix}, \\ B_\sigma(\boldsymbol{\alpha}) &= \begin{pmatrix} 0 \\ M_\theta^{-1} B_\sigma \end{pmatrix}, \end{aligned} \quad (8)$$

where $H_V = \nabla_{\mathbf{q}}^2 V(\mathbf{q}^*)$ is the Hessian of the potential energy evaluated at the equilibrium, and all quantities are functions of $\boldsymbol{\alpha}$ through $\mathbf{q}^*(\boldsymbol{\alpha})$.

When specialized to the quadrotor with $\sigma = \mathbf{0}$ (no contacts, no constraints), $J_\sigma = 0$ and $\boldsymbol{\lambda} = 0$, and (8) reduces to the block structure of [1, Eq. (5)] with $G(\boldsymbol{\alpha})$ playing the role of $-M^{-1} H_V$.

Training. The EL-Net for each mode σ is trained on trajectory data $\{(\mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{u}_t, \ddot{\mathbf{q}}_t)\}$ collected from simulation or hardware. The loss is the prediction error on $\ddot{\mathbf{q}}$:

$$\mathcal{L}_{\text{EL}}(\theta) = \sum_t \left\| \ddot{\mathbf{q}}_t - M_\theta(\mathbf{q}_t)^{-1} (B_\sigma \mathbf{u}_t - C_\theta \dot{\mathbf{q}}_t - \nabla V(\mathbf{q}_t)) \right\|^2. \quad (9)$$

The Christoffel matrix C_θ and the potential gradient ∇V are obtained by automatic differentiation through M_θ and V_θ , respectively. Because V_{grav} is known analytically, the network V_θ only needs to learn the internal potential—a simpler function.

D. DARE Cache and Gain Interpolation

This component is identical in structure to [1], but applied per mode σ .

Offline. For each reachable mode $\sigma \in \mathcal{I}$ and a grid $\{\boldsymbol{\alpha}^{(j)}\}_{j=1}^{M_\sigma}$ of scheduling variables:

- 1) Query the EL-Net to obtain $A_\sigma(\boldsymbol{\alpha}^{(j)})$, $B_\sigma(\boldsymbol{\alpha}^{(j)})$.
- 2) Discretize: $A_d = I + A_\sigma \Delta t$, $B_d = B_\sigma \Delta t$.
- 3) Solve the augmented DARE [2]:

$$P = Q_\rho + A_d^\top P (A_d - B_d (R_\rho + B_d^\top P B_d)^{-1} B_d^\top P A_d), \quad (10)$$

where $Q_\rho = Q + \rho I$, $R_\rho = R + \rho I$ are ADMM-augmented cost matrices.

- 4) Compute gain $K_\infty^{(j)} = (R_\rho + B_d^\top P B_d)^{-1} B_d^\top P A_d$ and cache $\{A_d, B_d, K_\infty, P, C_1, C_2\}^{(\sigma, j)}$.

Algorithm 1 Online: Assembled Switched LPV Template MPC

Require: State $\mathbf{x}_t$, command $\mathbf{s}$, surface $\mathcal{S}$, DARE caches

```

1: — Layer 2: Footstep Planner —
2:  $\{\mathbf{p}_i^{\text{des}}(k)\}_{k=0}^{N-1} \leftarrow \pi_\psi(\mathbf{x}_t, \mathbf{s}, \mathcal{S})$ 
3: for  $k = 0, \dots, N-1$  do
4:    $\sigma(k) \leftarrow \text{derive from } \phi(\mathbf{p}_i^{\text{des}}(k))$  (5)
5:    $\alpha(k) \leftarrow \text{extract from } (\mathbf{x}_t, \mathbf{p}^{\text{des}}(k), \sigma(k))$  Def. 4
6: end for
7: — Layer 3: Assemble & Solve MPC —
8: for  $k = 0, \dots, N-1$  do
9:    $(A_d, B_d, K_\infty, \dots)^{(k)} \leftarrow \text{CacheLookup}(\sigma(k), \alpha(k))$ 
10:   $(\mathbf{x}^*, \mathbf{u}^*)^{(k)} \leftarrow \text{Equilibrium}(\sigma(k), \alpha(k))$ 
11: end for
12:  $\mathbf{x}_{\text{ref}} \leftarrow \text{from } \mathbf{s}$  user command  $\rightarrow$  reference
13: Run  $K$  ADMM iterations with time-varying
    $\{A_d, B_d, K_\infty\}^{(k)}$ 
14:  $\mathbf{u}_t \leftarrow \mathbf{u}^{*(0)} + \delta \mathbf{u}_0$ 

```

MPC problem assembly (Algorithm 1). The MPC problem is not passed through a monolithic solver. At each control step, the complete time-varying quadratic program is *assembled* from four independently sourced components:

Source	Provides	Role in QP
User command $\mathbf{s}$	$\mathbf{x}_{\text{ref}}$	Reference trajectory
Footstep Planner	$\sigma(k), \alpha(k)$	Constraints, scheduling
EL-Net $_{\sigma(k)}$	$A_d(k), B_d(k)$	Dynamics matrices
DARE cache	$K_\infty(k), P(k)$	Feedback gains

Because the Planner outputs a time-stepped sequence, the mode $\sigma(k)$ and scheduling variable $\alpha(k)$ can vary *within* the MPC horizon. The resulting QP has time-varying dynamics—the matrices (A_d, B_d) change at each timestep k —but each per-step ADMM pass still uses only cached matrix-vector products.

Zero-iteration variant. As in [1], the gain can be approximated by first-order Taylor expansion around a reference α_0 within each mode:

$$K_\sigma(\alpha) \approx K_\sigma(\alpha_0) + \sum_{i=1}^{d_\sigma} \delta \alpha_i \frac{\partial K_\sigma}{\partial \alpha_i}, \quad (11)$$

where the gain sensitivities $\partial_i K_\sigma$ are computed offline by central differences. The online cost is then $O(n \cdot m \cdot d_\sigma)$ multiply-accumulate operations.

E. Training Pipeline

The three learned components—EL-Nets, DARE caches, and Footstep Planner—are trained in a three-stage pipeline (Algorithm 2). The stages are sequential: the DARE cache requires trained EL-Nets, and the Planner reward requires a functional DARE-cached controller. An optional fourth stage performs within-mode joint fine-tuning via implicit DARE differentiation.

Stage 1: EL-Net (supervised, per mode). For each mode σ , trajectory data $\mathcal{D}_\sigma = \{(\mathbf{q}_t, \dot{\mathbf{q}}_t, \mathbf{u}_t, \ddot{\mathbf{q}}_t)\}$ is collected from

a MuJoCo simulator [17] operating in mode σ . The EL-Net parameters θ_σ are updated by gradient descent on (9). The gradient flows through the Christoffel construction (6) and the potential V_θ via automatic differentiation. The *Push–Prove–Pull* structure is:

$$\begin{aligned}
&\text{Push: } \theta \rightarrow (M_\theta, V_\theta, C_\theta) \rightarrow \hat{\mathbf{q}}, \\
&\text{Prove: } \mathcal{L}_{\text{EL}} = \|\ddot{\mathbf{q}} - \hat{\mathbf{q}}\|^2, \\
&\text{Pull: } \theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{\text{EL}}.
\end{aligned} \quad (12)$$

Stage 2: DARE cache (numerical). Using the trained EL-Nets, the DARE cache is built for each mode σ over a grid of scheduling variables as described in Section III-D. No learning; pure numerical linear algebra.

Stage 3: Footstep Planner (RL). With EL-Nets and DARE caches frozen, the Planner π_ψ is trained by sampling N foot-placement sequences, rolling them forward through the DARE-cached MPC in MuJoCo, and reweighting by trajectory cost via Boltzmann weights [18]:

$$w^{(n)} = \frac{\exp(-L^{(n)}/\lambda)}{\sum_{n'} \exp(-L^{(n')}/\lambda)}, \quad (13)$$

where $L^{(n)} = \sum_t (\|\mathbf{x}_t - \mathbf{x}_{\text{ref}}\|_Q^2 + \|\mathbf{u}_t\|_R^2)$ is the tracking cost of rollout n and $\lambda > 0$ is a temperature. The Planner parameters are updated via:

$$\psi \leftarrow \psi - \eta_\psi \sum_{n=1}^N w^{(n)} \nabla_\psi \log \pi_\psi(\mathbf{p}^{(n)} | \mathbf{x}, \mathbf{s}). \quad (14)$$

Pre-defined gaits (trot, walk, gallop) initialize π_ψ via behavioral cloning before RL fine-tuning.

Stage 4 (optional): Within-mode fine-tuning. Within a fixed mode σ , the full chain from EL-Net through DARE to tracking cost is differentiable:

$$\begin{aligned}
&\text{Push: } \theta \rightarrow (A_\sigma(\alpha), B_\sigma(\alpha)), \\
&\text{Prove: DARE} \rightarrow K_\infty \rightarrow \mathbf{u} \rightarrow \mathcal{L}_{\text{track}}, \\
&\text{Pull: } \theta \leftarrow \theta - \eta \partial \mathcal{L}_{\text{track}} / \partial \theta \text{ (Lyap. (18))}.
\end{aligned} \quad (15)$$

This fine-tunes EL-Net weights to minimize downstream MPC tracking error, not just acceleration prediction error. The DARE cache is rebuilt after each parameter update.

Why not end-to-end? The mode derivation (5) involves a hard threshold ($\phi = 0$), blocking gradient flow from the DARE cache to the Planner. Remark 2 explains why softening this threshold is not a valid alternative. Within each mode, however, the pipeline from EL-Net through DARE to tracking cost *is* differentiable (Stage 4), enabling joint fine-tuning of the EL-Net and cost matrices Q, R .

IV. ANALYSIS

A. Generalized Equivalence Chain

Theorem 1 (Equivalence Chain). *For a switched LPV system, the following controllers form a hierarchy of decreasing online*

Algorithm 2 Offline Training (MuJoCo Simulator)**Require:** Simulator f_{sim} , modes $\mathcal{I}$, α -grids, Q, R, ρ

```

1: — Stage 1: EL-Net (per mode) —
2: for each  $\sigma \in \mathcal{I}$  do
3:    $\mathcal{D}_\sigma \leftarrow \text{collect } (q, \dot{q}, u, \ddot{q}) \text{ from } f_{\text{sim}} \text{ in mode } \sigma$ 
4:   for epoch = 1, ...,  $E_1$  do
5:      $\theta_\sigma \leftarrow \theta_\sigma - \eta_1 \nabla_{\theta_\sigma} \mathcal{L}_{\text{EL}}$  (9)
6:   end for
7: end for
8: — Stage 2: DARE cache —
9: for each  $\sigma \in \mathcal{I}$ , grid point  $\alpha^{(j)}$  do
10:   $(A_d, B_d) \leftarrow \text{discretize EL-Net}_\sigma(\alpha^{(j)})$ 
11:  DARE (10)  $\rightarrow (K_\infty, P)^{(\sigma, j)}$ 
12:   $\partial_i K_\sigma \leftarrow \text{central differences}$ 
13: end for
14: — Stage 3: Footstep Planner (RL) —
15: Init  $\pi_\psi \leftarrow \text{behavioral clone of pre-defined gait}$ 
16: for iter = 1, ...,  $I_3$  do
17:   for  $n = 1, \dots, N$  do
18:      $\{p_i^{(n)}\} \leftarrow \pi_\psi(x, s, S) + \epsilon_n$ 
19:     Roll out in  $f_{\text{sim}}$  with Alg. 1
20:      $L^{(n)} \leftarrow \text{trajectory cost}$ 
21:   end for
22:    $w^{(n)} \leftarrow \text{Boltzmann (13)}$ 
23:    $\psi \leftarrow \psi - \eta_\psi \sum_n w^{(n)} \nabla_\psi \log \pi_\psi$  (14)
24: end for
25: — Stage 4 (opt): Fine-tune per mode —
26: for each  $\sigma$ , epoch = 1, ...,  $E_4$  do
27:    $\nabla_\theta \mathcal{L}_{\text{track}} \leftarrow (24) \text{ via Lyap. (18)}$ 
28:    $\theta_\sigma \leftarrow \theta_\sigma - \eta_4 \nabla_\theta \mathcal{L}_{\text{track}}$ ; rebuild cache
29: end for
30: return  $\{\theta_\sigma\}$ , caches,  $\psi$ 

```

computation, with each step introducing a bounded approximation:

$$\begin{aligned} & \text{Full TO} \xrightarrow{\text{fix } \sigma} \text{Mode-Given TO} \xrightarrow{\text{cache DARE}} \\ & \text{Template MPC} \xrightarrow{K \rightarrow 0} \text{Analytic Template} \xrightarrow{\text{1st ord.}} \text{Gain Sched.} \end{aligned} \quad (16)$$

Proof. We trace each arrow.

(i) *Full TO $\rightarrow$ Mode-Given TO.* Full trajectory optimization over both σ and u (e.g., contact-implicit [4]) jointly selects the mode sequence and control. Fixing σ via the Footstep Planner yields a smooth optimal control problem within the selected mode. The approximation error is bounded by the Planner's sub-optimality in mode selection.

(ii) *Mode-Given TO $\rightarrow$ Template MPC.* Within mode σ , online relinearization at each timestep followed by DARE solve gives the optimal LQR gain $K_\sigma(\alpha_t)$. Replacing this with a cached DARE at the nearest grid point $\alpha^{(j)}$ introduces an error $\|K_\sigma(\alpha_t) - K_\sigma(\alpha^{(j)})\|$, bounded by Proposition 4 and controlled by grid density.

(iii) *Template MPC $\rightarrow$ Analytic Template.* Setting the number of ADMM iterations $K = 0$ (with zero warm-start)

Algorithm 3 Online Deployment (Real Robot)**Require:** Trained π_ψ , EL-Nets $\{\theta_\sigma\}$, DARE caches, sensors

```

1: Load caches and weights to onboard memory
2:  $\mathcal{D}_{\text{on}} \leftarrow \emptyset$  (adaptation buffer)
3: loop
4:    $s \leftarrow \text{user body-pose command}$  (6 numbers)
5:    $x_t \leftarrow \text{IMU} + \text{joint encoders}$ 
6:    $S \leftarrow \text{depth camera}$  (terrain SDF)
7:   — Layer 2: Plan —
8:    $\{p_i^{\text{des}}(k)\}_{k=0}^{N-1} \leftarrow \pi_\psi(x_t, s, S)$ 
9:   for  $k = 0, \dots, N-1$  do
10:     $\sigma(k), \alpha(k) \leftarrow \text{derive from } \phi(p_i^{\text{des}}(k))$ 
11:   end for
12:   — Layer 3: Assemble & Solve (Alg. 1) —
13:    $u_t \leftarrow \text{DARE-cached ADMM with time-varying } \{\sigma(k), \alpha(k)\}$ 
14:   — Layer 4: Execute —
15:   Apply  $u_t$  to motors
16:   — Online Adaptation (periodic) —
17:    $\mathcal{D}_{\text{on}} \leftarrow \mathcal{D}_{\text{on}} \cup \{(q_t, \dot{q}_t, u_t, \ddot{q}_t)\}$ 
18:   if  $|\mathcal{D}_{\text{on}}| \geq B_{\text{adapt}}$  then
19:      $\theta_{\sigma_t} \leftarrow \theta_{\sigma_t} - \eta_a \nabla_{\theta} \mathcal{L}_{\text{EL}}(\mathcal{D}_{\text{on}})$ 
20:     Rebuild cache for current  $(\sigma, \alpha)$ 
21:      $\mathcal{D}_{\text{on}} \leftarrow \emptyset$ 
22:   end if
23: end loop

```

TABLE II

ONLINE COMPUTATIONAL COMPLEXITY OF EACH CONTROLLER VARIANT.
 n : STATE DIMENSION, m : INPUT DIMENSION, K : ADMM ITERATIONS,
 n_π : PLANNER NETWORK SIZE, d : SCHEDULING DIMENSION.

Variant	Online cost	Constraints
Full TO	$O(n^3 N)$	Yes
Mode-Given TO	$O(n^3) + K \cdot O(n^2)$	Yes
Template MPC	$n_\pi + K \cdot O(n^2)$	Yes
Analytic Templ.	$n_\pi + O(n \cdot m \cdot d)$	Clip only

yields $u_0 = u^* - K_\infty(\alpha) \delta x$, which is the unconstrained LQR feedback. The error relative to converged ADMM is the constraint violation [1].

(iv) *Analytic Template $\rightarrow$ Gain Schedule.* Replacing $K_\infty(\alpha)$ by the first-order interpolation (11) introduces the Taylor remainder error (Proposition 4). $\square$

B. Sufficient Conditions

Proposition 3 (Applicability Conditions). *The switched LPV Template MPC framework is applicable when:*

- 1) **Finite modes:** The system has a finite number of reachable contact modes $|\mathcal{I}| < \infty$.
- 2) **Intra-mode smoothness:** Within each mode σ , the dynamics $A_\sigma(\alpha)$, $B_\sigma(\alpha)$ are $\mathcal{C}^2$ functions of α .
- 3) **Low-dimensional scheduling:** $d_\sigma \ll n$ for all $\sigma \in \mathcal{I}$.
- 4) **Stabilizability:** For each (σ, α) , the pair $(A_\sigma(\alpha), B_\sigma(\alpha))$ is stabilizable, so the DARE has a unique stabilizing solution.

5) **Gain smoothness:** $K_\sigma(\alpha)$ is $\mathcal{C}^1$ in α with bounded gradient, so first-order interpolation has controlled error.

Condition (1) excludes continuous contact surfaces (e.g., rolling) without discretization. Condition (3) is what makes caching feasible: a grid over $\mathbb{R}^d$ with $d = 6$ is practical; a grid over $\mathbb{R}^{36}$ (full quadruped state) is not. Condition (4) is inherited from standard LQR theory. Condition (5) follows from condition (2) by the implicit function theorem applied to the DARE.

C. Gain Interpolation Error

Proposition 4 (Interpolation Error Bound). *Let $K_\sigma : \mathbb{R}^{d_\sigma} \rightarrow \mathbb{R}^{m \times n}$ be the DARE gain map for mode σ , assumed $\mathcal{C}^2$. Then the first-order interpolation error satisfies*

$$\|K_\sigma(\alpha) - K_\sigma(\alpha_0) - \nabla_\alpha K_\sigma(\alpha_0)^\top (\alpha - \alpha_0)\|_F \leq \frac{\beta_\sigma}{2} \|\alpha - \alpha_0\|^2, \quad (17)$$

where $\beta_\sigma = \sup_\alpha \|\nabla_\alpha^2 K_\sigma(\alpha)\|$ is the maximum Hessian norm over the scheduling domain.

Proof. This is the standard multivariate Taylor remainder theorem applied to the matrix-valued function K_σ . The $\mathcal{C}^2$ regularity is guaranteed by Proposition 3(2,5). $\square$

For the quadrotor ($d = 4$), [1] measured a mean relative error of 4% over the tested range. For a quadruped in trot ($d = 6$), we expect comparable accuracy for moderate COM excursions, with the error growing quadratically for large deviations from the reference α_0 .

D. DARE Differentiability

Proposition 5 (Implicit Differentiation of DARE). *The DARE solution P as a function of (A, B, Q, R) is differentiable wherever it exists and is unique. The gradient $\partial P / \partial A$ is the solution of a discrete Lyapunov equation:*

$$\frac{\partial P}{\partial A} = A_{\text{cl}}^\top \frac{\partial P}{\partial A} A_{\text{cl}} + F, \quad (18)$$

where $A_{\text{cl}} = A - BK_\infty$ is the closed-loop matrix and F is a forcing term depending on P , K_∞ , and the perturbation direction.

This enables backpropagation from the MPC tracking cost through the DARE cache to the EL-Net parameters θ —within each mode. The gradient flow is:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial K} \frac{\partial K}{\partial P} \frac{\partial P}{\partial (A, B)} \frac{\partial (A, B)}{\partial \theta}.$$

The chain terminates at the EL-Net; it does not cross mode boundaries (which require RL, as discussed in Section III-B).

V. INSTANTIATION EXAMPLES

A. Quadrotor: Recovering Paper 1

The quadrotor has $C = 0$ contact points, so $\mathcal{I} = \{\mathbf{0}\}$ (single mode) and $\mathcal{S} = \emptyset$. The scheduling variable is $\alpha = (a_x, a_y, a_z, \psi)$ from differential flatness [1], not from the general formula $d_\sigma = 2|\sigma| + 2 = 2$.

TABLE III
QUADROTOR FIGURE-EIGHT TRACKING [1]: THE GENERAL FRAMEWORK RECOVERS PAPER 1 RESULTS.

Variant	Avg. error (m)	ADMM iters	Speedup
Online Relin	0.035	1023	1×
Flat Template	0.035	714	29×
Analytic Templ.	0.036	0	85×

The EL-Net is replaced by the analytic flat linearization: $A(\alpha)$, $B(\alpha)$ are closed-form trigonometric expressions [1, Eq. (5)]. The DARE cache is built over a 4D grid of α values. The Footstep Planner is absent (no contacts to schedule).

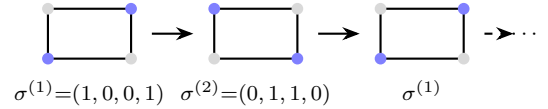
The full equivalence chain (16) reduces to the quadrotor-specific chain of [1]:

$$\text{Online Relin} \xrightarrow{\text{cache}} \text{Flat Template} \xrightarrow{K \rightarrow 0} \text{Analytic Template} \xrightarrow{\text{1st ord.}} \text{Gain Sched.}$$

Table III reproduces the key results from [1] as validation.

B. Quadruped Trot: Worked Example

Consider a quadruped with $C = 4$ feet. A trotting gait cycles through contact modes corresponding to alternating diagonal pairs:



In each phase, $|\sigma| = 2$ (two feet on ground), so $d_\sigma = 2 \cdot 2 + 2 = 6$. The scheduling variable is:

$$\alpha_\sigma = \left(\underbrace{(p_i - p_{\text{com}})_{xy}}_{\text{foot 1}}, \underbrace{(p_j - p_{\text{com}})_{xy}}_{\text{foot 2}}, \underbrace{s_{xy}}_{\text{COM skew}} \right) \in \mathbb{R}^6, \quad (19)$$

where i, j are the indices of the active feet and s_{xy} is the COM position relative to the midpoint of the two active feet, projected onto the ground plane.

Storage budget. For the zero-iteration variant, each mode requires $K_\sigma(\alpha_0) \in \mathbb{R}^{m \times n}$ plus d_σ sensitivity matrices $\partial_i K_\sigma \in \mathbb{R}^{m \times n}$. With $m = 12$ (joint torques), $n = 24$ (generalized coordinates + velocities), $d_\sigma = 6$:

$$(1 + 6) \times 12 \times 24 = 2016 \text{ floats per mode.}$$

For a trot gait with 2 distinct modes: $2 \times 2016 = 4032$ floats ≈ 16 kB (single precision).

Comparison with WBC. The standard WBC pipeline for this system would use: (i) a gait scheduler that prescribes $\sigma^{(1)} \rightarrow \sigma^{(2)} \rightarrow \sigma^{(1)} \rightarrow \dots$ at fixed timing; (ii) a rigid-body dynamics model for the contact-consistent dynamics; and (iii) a QP solver (e.g., qpOASES) to compute torques online. Our framework replaces (i) with the Footstep Planner (adaptive timing and placement), (ii) with EL-Nets (learned, structured), and (iii) with DARE-cached feedback (offline solve). The pre-defined trot gait can initialize the Planner training.

VI. DISCUSSION

Exploration vs. exploitation. The architecture separates two fundamentally different computational problems:

- *Exploration* (Footstep Planner): Which contact sequence leads to a good trajectory? This is combinatorial (2^C modes per step) and requires search or learning. The Planner amortizes this search into a single forward pass—analogue to AlphaGo’s policy network, which replaces game-tree search with pattern matching [14].
- *Exploitation* (EL-Net + DARE cache): Given the contact sequence, what is the optimal feedback? This is continuous, smooth, and amenable to classical control theory. The DARE cache enables constant-time lookup—analogue to AlphaGo’s value network, which replaces rollout evaluation with fast inference.

Limitations.

- 1) *Mode scaling*: $|Z| \leq 2^C$ grows exponentially with the number of contact points. For dexterous manipulation ($C \geq 10$), mode clustering or hierarchical decomposition is needed.
- 2) *Two-stage training*: The Planner and EL-Nets cannot be trained end-to-end (Remark 2). This increases engineering complexity compared to monolithic RL policies.
- 3) *Mode transition dynamics*: The current formulation assumes instantaneous mode switching. Impact dynamics and guard conditions at mode boundaries are not modeled.
- 4) *Gain interpolation accuracy*: The first-order Taylor approximation (11) degrades for large $\|\alpha - \alpha_0\|$. For $d_\sigma = 6$ (quadruped trot), a second-order expansion or denser grid may be needed.

Minimal specification. In the standard WBC pipeline, the engineer must provide two objects in addition to the user command: a gait table (specifying which feet contact the ground and when) and a dynamics model (specifying inertial parameters, joint friction, and contact geometry). Both require expert knowledge and careful tuning. In our framework, neither is required. The Footstep Planner learns the contact schedule from experience; the EL-Net learns the dynamics from data. The only human input is $s \in \mathbb{R}^6$: six numbers specifying where the body should be. Everything else is either learned or derived from classical control theory. This is the sharpest possible form of the WBC decomposition: retain the layered structure (which is architecturally sound) while eliminating all pre-specified domain knowledge.

What is classical vs. what is new. Gain scheduling [8], LPV control [9], Euler–Lagrangian learning [5], and Riccati sensitivity [10], [11] are individually well established. Our contribution is their integration into a modular four-layer architecture for contact-rich MPC: (i) the state decomposition (COM skew + relative foot positions) that yields an exactly determined, low-dimensional scheduling variable; (ii) the time-stepped Footstep Planner that outputs a complete contact schedule with geometry over the MPC horizon; (iii) the assembled MPC formulation where each component of the QP

is sourced independently; (iv) the identification of Remark 2—that soft mode mixing is physically meaningless—as the fundamental reason for two-stage training.

VII. CONCLUSION

We presented a four-layer architecture for real-time MPC of contact-rich robotic systems in which the only external specification is a six-dimensional body-pose command. The Footstep Planner (Layer 2) outputs a time-stepped contact schedule over the MPC horizon; the EL-Nets provide structured per-mode dynamics; the DARE cache enables constant-time feedback. The MPC problem at each timestep is assembled from these independently sourced components, not solved monolithically.

The quadrotor result of [1] is recovered as a single-mode special case with analytic dynamics. For quadruped trotting, the scheduling variable has dimension $d = 6$, yielding a storage budget of ~ 16 kB and practical computation requirements.

Future work includes: hardware deployment on the Unitree Go2 platform, extension to dexterous manipulation via mode clustering, online adaptation of EL-Net weights via meta-learning, and formal switched-system stability analysis with dwell-time constraints.

Yes, this is what you need to train a robot: a completed Yang–Mills theory. And this is what was missing.

APPENDIX

We derive the linearized dynamics (8) from the EL equations of motion (7).

Consider an equilibrium $(q^*, \dot{q}^* = 0, u^*)$ within mode σ . Define perturbations $\delta q = q - q^*$, $\delta \dot{q} = \dot{q}$, $\delta u = u - u^*$.

The EL equations at the equilibrium satisfy:

$$\nabla_q V(q^*) = B_\sigma u^* + J_\sigma(q^*)^\top \lambda^*. \quad (20)$$

Linearizing (7) about this equilibrium and subtracting (20):

$$M^* \delta \ddot{q} + C^* \delta \dot{q} + H_V \delta q = B_\sigma \delta u + \underbrace{\frac{\partial}{\partial q} (J_\sigma^\top \lambda)}_{\text{contact stiffness}} \Big|_* \delta q, \quad (21)$$

where $M^* = M_\theta(q^*)$, $C^* = C_\theta(q^*, 0)$ (which vanishes at $\dot{q}^* = 0$ by the Christoffel construction (6)), and $H_V = \nabla_q^2 V(q^*)$.

For rigid contacts (infinite stiffness), the contact term is absorbed into the holonomic constraint $J_\sigma \delta q = 0$, and the linearization reduces to the constrained form. In the unconstrained generalized coordinates (after projecting out the constrained directions), we obtain the state-space form:

$$\frac{d}{dt} \begin{pmatrix} \delta q \\ \delta \dot{q} \end{pmatrix} = \underbrace{\begin{pmatrix} 0 & I \\ -(M^*)^{-1} H_V & 0 \end{pmatrix}}_{A_\sigma(\alpha)} \begin{pmatrix} \delta q \\ \delta \dot{q} \end{pmatrix} + \underbrace{\begin{pmatrix} 0 \\ (M^*)^{-1} B_\sigma \end{pmatrix}}_{B_\sigma(\alpha)} \delta u,$$

which is (8) with $C^* = 0$ at the equilibrium.

When specialized to the quadrotor: $M^* = mI_3$ (scalar mass, no rotational coupling in translational dynamics), $H_V = -m \partial^2 V_{\text{grav}} / \partial q^2 = G(\alpha)$ (the gravity coupling matrix of [1,

Eq. (5)), and $B_\sigma = k_t \cdot [\text{thrust direction}]$, recovering the quadrotor-specific block structure.

The DARE (10) defines P implicitly as a function of (A, B, Q, R) . Define the residual:

$$F(P, A, B) \triangleq P - Q_\rho - A^\top P(A - BK(P)) = 0, \quad (22)$$

where $K(P) = (R_\rho + B^\top PB)^{-1} B^\top PA$.

By the implicit function theorem, $\partial P / \partial A$ satisfies:

$$\frac{\partial F}{\partial P} \cdot \frac{\partial P}{\partial A} = -\frac{\partial F}{\partial A}.$$

The operator $\partial F / \partial P$ acts on a perturbation ΔP as:

$$\frac{\partial F}{\partial P}[\Delta P] = \Delta P - A_{\text{cl}}^\top \Delta P A_{\text{cl}},$$

where $A_{\text{cl}} = A - BK$ is the closed-loop matrix (stable by the DARE solution).

Therefore, $\partial P / \partial A$ is the solution of the *discrete Lyapunov equation*:

$$\Delta P - A_{\text{cl}}^\top \Delta P A_{\text{cl}} = \frac{\partial F}{\partial A}, \quad (23)$$

which has a unique solution because A_{cl} is stable (all eigenvalues inside the unit circle). This is solvable in $O(n^3)$ by the Bartels–Stewart algorithm [15].

The complete gradient chain for EL-Net training is:

$$\frac{\partial \mathcal{L}}{\partial \theta} = \frac{\partial \mathcal{L}}{\partial \mathbf{u}} \frac{\partial \mathbf{u}}{\partial K} \frac{\partial K}{\partial P} \frac{\partial P}{\partial(A_d, B_d)} \frac{\partial(A_d, B_d)}{\partial(A_c, B_c)} \frac{\partial(A_c, B_c)}{\partial \theta}, \quad (24)$$

where $\partial(A_d, B_d) / \partial(A_c, B_c)$ is the discretization Jacobian ($\Delta t \cdot I$ for forward Euler), and $\partial(A_c, B_c) / \partial \theta$ is obtained by automatic differentiation through the EL-Net.

This gradient chain enables fine-tuning EL-Net weights to minimize downstream tracking cost, *within* a fixed contact mode. It does *not* cross mode boundaries—those require the RL-based Footstep Planner training of Section III-B.

REFERENCES

- [1] J. Zhang, “TinyMPC with differential flatness: α -parameterized templates for efficient quadrotor control,” 2025, preprint.
- [2] A. Alavilli, K. Nguyen, S. Schoedel, B. Plancher, and Z. Manchester, “TinyMPC: Model-predictive control on resource-constrained microcontrollers,” in *Proc. IEEE ICRA*, 2024.
- [3] S. Kim, D. Kim, and S. Kim, “Highly dynamic quadruped locomotion via whole-body impulse control and model predictive control,” 2019, arXiv:1909.06586.
- [4] M. Posa, C. Cantu, and R. Tedrake, “A direct method for trajectory optimization of rigid bodies through contact,” *Int. J. Robotics Research*, vol. 33, no. 1, pp. 69–81, 2014.
- [5] M. Lutter, C. Ritter, and J. Peters, “Deep Lagrangian Networks: Using physics as model prior for deep learning,” in *Proc. ICLR*, 2019.
- [6] D. Mellinger and V. Kumar, “Minimum snap trajectory generation and control for quadrotors,” in *Proc. IEEE ICRA*, 2011, pp. 2520–2525.
- [7] R. M. Murray, M. Rathinam, and W. Sluis, “Differential flatness of mechanical control systems,” in *Proc. ASME IMECE*, 1995.
- [8] W. J. Rugh and J. S. Shamma, “Research on gain scheduling,” *Automatica*, vol. 36, no. 10, pp. 1401–1425, 2000.
- [9] J. S. Shamma and M. Athans, “Guaranteed properties of gain scheduled control for linear parameter-varying plants,” *Automatica*, vol. 27, no. 3, pp. 559–564, 1991.
- [10] G. A. Hewer, “An iterative technique for the computation of the steady state gains for the discrete optimal regulator,” *IEEE Trans. Automatic Control*, vol. 16, no. 4, pp. 382–384, 1971.

- [11] M. M. Konstantinov, P. Hr. Petkov, and N. D. Christov, “Perturbation analysis of the discrete Riccati equation,” *Kybernetika*, vol. 29, no. 1, pp. 18–29, 1993.
- [12] R. J. Williams, “Simple statistical gradient-following algorithms for connectionist reinforcement learning,” *Machine Learning*, vol. 8, pp. 229–256, 1992.
- [13] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, “Proximal policy optimization algorithms,” 2017, arXiv:1707.06347.
- [14] D. Silver *et al.*, “Mastering the game of Go with deep neural networks and tree search,” *Nature*, vol. 529, pp. 484–489, 2016.
- [15] R. H. Bartels and G. W. Stewart, “Solution of the matrix equation $AX + XB = C$,” *Comm. ACM*, vol. 15, no. 9, pp. 820–826, 1972.
- [16] A. Bemporad, M. Morari, V. Dua, and E. N. Pistikopoulos, “The explicit linear quadratic regulator for constrained systems,” *Automatica*, vol. 38, no. 1, pp. 3–20, 2002.
- [17] E. Todorov, T. Erez, and Y. Tassa, “MuJoCo: A physics engine for model-based control,” in *Proc. IEEE/RSJ IROS*, 2012, pp. 5026–5033.
- [18] G. Williams, N. Wagener, B. Goldfain, P. Drews, J. M. Rehg, B. Boots, and E. A. Theodorou, “Information theoretic MPC for model-based reinforcement learning,” in *Proc. IEEE ICRA*, 2017, pp. 1714–1721.